

MEDICART

두 대의 AMR로 간호사 투약과 순회 문진을 수행하는
병동 보조 로봇 시스템 — 최종 보고서

CONTENTS

목 차

01 프로젝트 개요

주제·배경 · 내용/컨셉 · 장비 · 구조 · 기대효과

02 팀 구성 및 역할

9인 · PM · AMR · Detection · DB/Monitor · 환경

03 수행 절차 및 방법

개발 프로세스 · 일정 계획 vs 실행

04 수행 경과

시스템 아키텍처 · Detection · AMR · Monitor · 통합테스트 · R&D

05 자체 평가 의견

성과 · 한계 · 향후 과제

핵심 키워드

ROS2 · Nav2 · Firebase RTDB · YOLO · OCR/QR · Next.js

PROJECT OVERVIEW · 주제 및 선정 배경

병동의 업무 과부하·오투약 리스크를 로봇으로

현장 문제 (Business Requirements)

- 01 간호사 **1인당 담당 환자 과다** → 단순 문의·순회에 시간 소모
- 02 수기 약품 관리 → **오투약 리스크**
- 03 외국인·고령 환자 증가 → **언어 장벽**으로 소통 지연
- 04 콜벨·수기 방식 → 호출 누락·우선순위 파악 불가

주제: 투약 보조 + 순회 문진을 자동화하는 **병동 보조 로봇**으로 간호 인력의 단순업무를 덜고 안전(이중 확인)을 높인다.



병원 현장 문제 인포그래픽 / 통계 이미지
간호사 업무 비중·오투약 통계 등 (팀 자료)

PROJECT CONTENT · 구현 내용 · 컨셉 · 훈련 연관성

2대의 AMR + 웹/앱으로 완성하는 병동 보조

시나리오 A · 간호사 투약 (robot6)

약품실 이동 → OCR 약품 검증 → 간호사 추종(85cm) → 환자 QR 재확인 → 복귀·도킹

시나리오 B · 순회 문진 (robot3)

병상 순회 → QR 배정환자 검증 → 문진 작성/부재증 → 다음 정차 → 복귀·도킹

훈련 내용과의 연관성

ROS2 Humble

Nav2 자율주행

SLAM/Localization

CV(YOLO·OCR·QR)

Depth/Homography

Firebase RTDB

Next.js-Flask 웹

STT-LLM



시스템 컨셉 한 장 이미지 (로봇 2대 + 병동 + 웹/앱)

여긴 전체 컨셉을 보여주는 합성/렌더 이미지가 들어가야 함

EQUIPMENT & ENVIRONMENT

활용 장비 및 개발 환경

하드웨어

구분	장비
AMR	TurtleBot4 ×2 (robot3·robot6) · iRobot Create3
센서	OAK-D (RGB-D) · 2D LIDAR (RPLIDAR)
외부	iPad 카메라(추적) · 키오스크 단말
컴퓨팅	3 PC (robot3·robot6·웹) + GPU

장비 사진 (TurtleBot4 · OAK-D · 키오스크)

실물 사진 자리

소프트웨어 스택

레이어	기술
로봇	ROS2 Humble · Nav2 · slam_toolbox · rclpy
인지	Ultralytics YOLO · EasyOCR · GCP Vision · cv2 QR
버스	Firebase Realtime DB (firebase-admin)
웹	Next.js 16 · Flask · Tailwind · vitest/pytest
R&D	Homography/BEV · Ollama(STT) · TensorRT

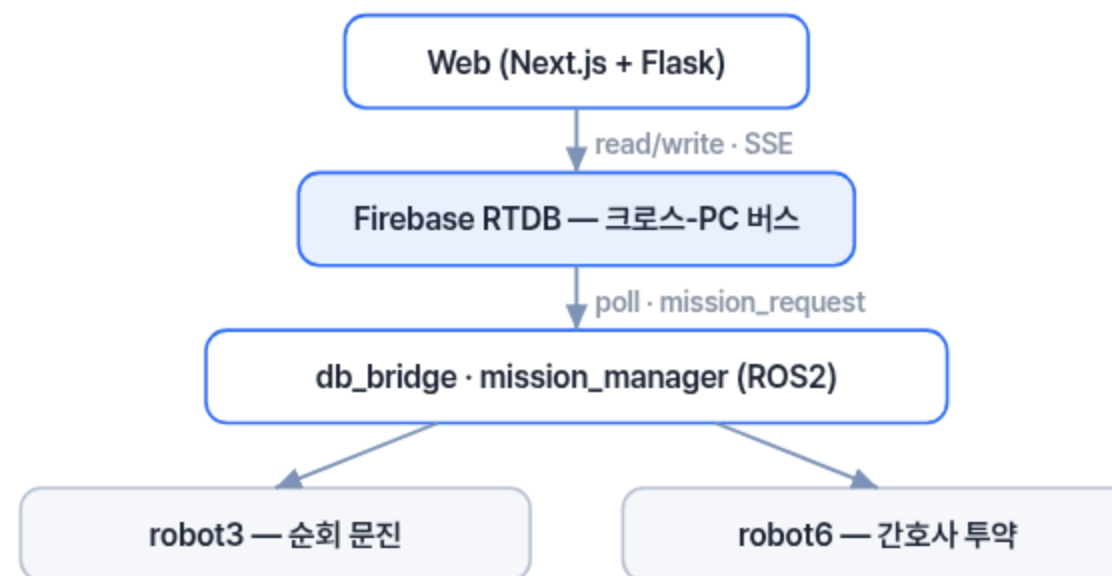
OS: Ubuntu 22.04 · 협업: Git/GitHub · cloudflared 터널

PROJECT STRUCTURE

프로젝트 구조 — 워크스페이스 & 시스템 한 장

레포 구조

```
# MediCart/
medicart_ws/  ROS2 (mission_manager·nurse_tracker·
               db_bridge·patient_identifier·
               obstacle·simulation·dashboard)
ocr_ws/       OCR detector (engines·node)
web/          frontend(Next.js)+backend(Flask)
common/       robot.env · maps(ninety/ward)
docs/         architecture·specs·presentation
```



시스템 한 장 — RTDB를 허브로 Web↔ROS2 결합 (상세는 4.1)

APPLICATION & EXPECTED EFFECT

활용 방안 및 기대 효과

간호 인력

단순 업무(순회·문의·기록) 자동화 → 핵심 의료 행위에 집중

환자 안전

OCR + QR 이중 확인으로 오투약 방지 · 투약 이력 추적

환자 경험

다국어 STT 자기서비스 → 대기 단축·만족도↑

현장 투입 시나리오

야간/인력 부족 시간대 순회·문진 자동 수행, 약품 전달 시 신원 재확인, 키오스크로 환자 호출·문의 중계 → **로봇-앱-간호사 3자 연동**으로 실사용 가능한 시스템.



기대효과 정량 인포그래픽 (시간 절감·정확도 등)

측정/추정 수치 그래프 자리

TEAM & ROLES

프로젝트 팀 구성 및 역할 (9인)

이름	역할	세부 내용
우지명	PM	프로젝트 관리
김재훈	DB & System Monitor	DB 구축·스키마, Web 호스팅·UI, DB↔ROS2 연동
박성현	System Architecture & Integration	아키텍처 설계·Nav2 map 구축, 시나리오 통합·테스트
최수진	AMR	투약 시나리오 통합, tracking 모델, 시나리오 플로우 정리
옥윤정	AMR	순찰 시나리오 통합, 아키텍처 설계, iPad 카메라, gazebo
전승엽	Detection	OCR/QR 모델 개발 및 Web&DB 연동
이건주	Detection · R&D	RGB 기반 Depth 추정 모델 개발
허찬영	Detection · R&D	OCR/QR 모델 개발 및 STT App 개발
조현규	환경 세팅	물리 맵 환경 세팅

PROCESS & METHOD

수행 절차 및 방법



협업·형상관리

Git/GitHub 브랜치 전략(main-integration·파트별), PR 리뷰. 문서는 docs/ 스펙·플랜으로 관리.

크로스-PC 통합

RTDB를 단일 버스로 두어 파트가 독립 개발 후 스키마로 결합 → 디커플링.

검증 전략

단위 대신 **시스템 통합 테스트** + 웹 레이어 자동화 테스트(별도).

SCHEDULE · PLAN vs ACTUAL

3.1 일정 계획 대비 실행



간트 차트 — 주차별 계획(상) vs 실제(하)

여긴 실제 팀 일정(설계·맵·AMR·Detection·Web·통합·시연)을 주차 단위 간트로 넣어야 함. 지연/조정 구간 강조.

설계·환경

SA·맵·Gazebo

인지/AMR

OCR/QR·tracking·Nav2

Web/DB

RTDB·대시보드

통합·시연

시나리오·테스트

왜 이렇게 설계했는가 — 3대 원칙

① RTDB 단일 버스

모든 크로스-PC 통신을 **Firebase RTDB** 하나로 — 미션·텔레메트리·핸드셰이크.

② AMR당 PC 분리

로봇마다 전용 PC → **ROS_DOMAIN/디스커버리** 격리·자원 분리.

③ cmd_vel 단독 소유

ModeArbiter가 cmd_vel을 단독 게이트 → 모드 충돌·폭주 방지.

설계 목표

로봇측 ROS 노드는 가급적 손대지 않고, **PC측 중재와 웹↔DB↔ROS** 디커플링으로 5개 모드를 안전하게 운용. 파트별 독립 개발 → RTDB 스키마로 통합.

RTDB hub

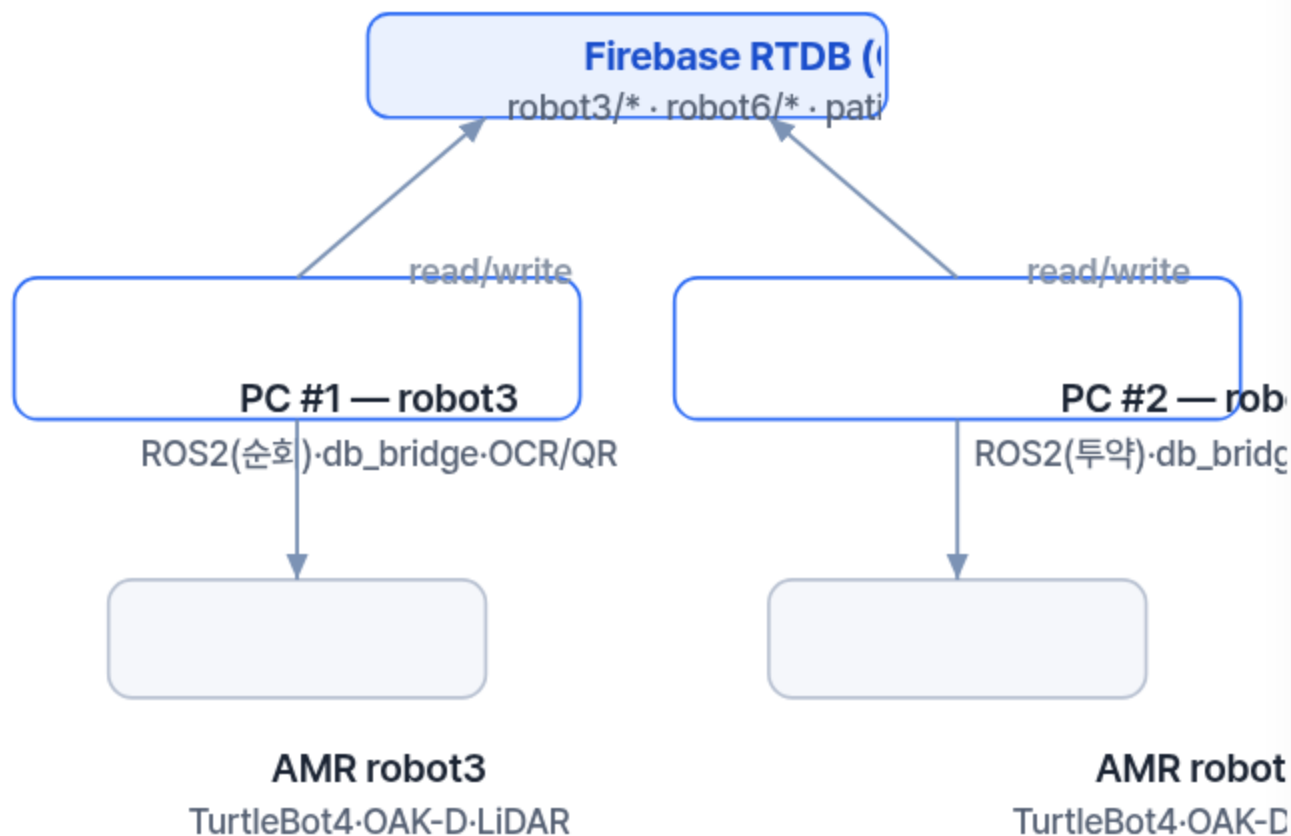
per-robot PC

priority preempt

robot ROS 무변경

DB / PC / AMR LEVEL

DB · PC · AMR 레벨 구조 + Why PC 분리

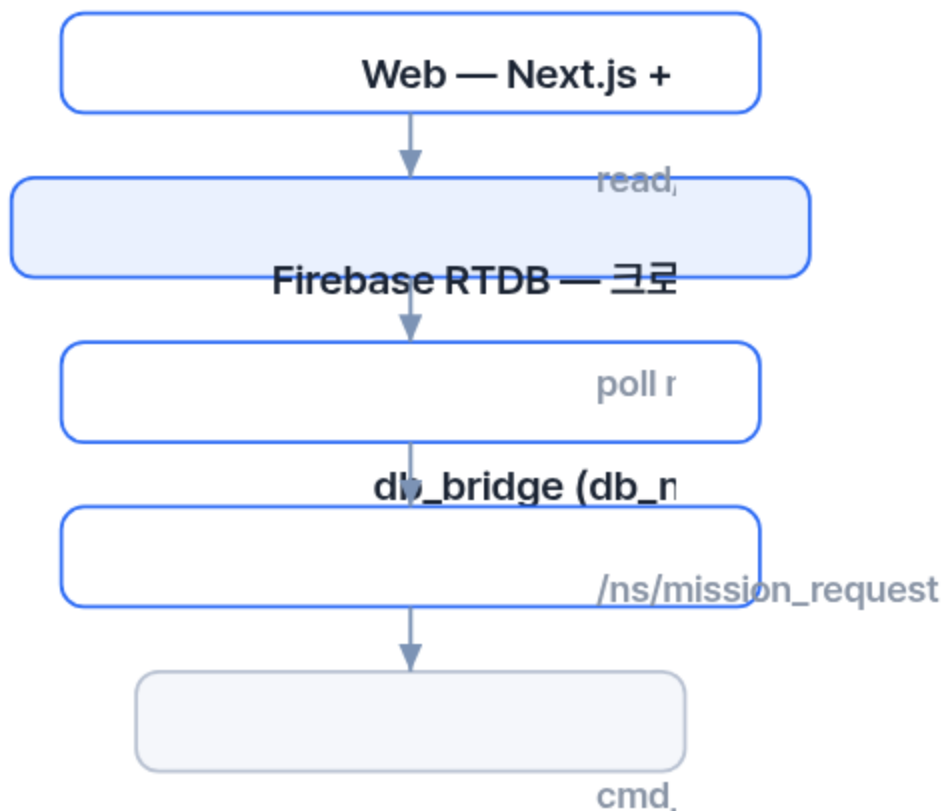


Why — AMR당 PC를 분리했는가

- **DDS 디스커버리 격리** — 두 로봇의 ROS2 트래픽 간섭·도메인 충돌 방지
- **자원 독립** — 인지(YOLO/OCR)·Nav2가 GPU/CPU를 로봇별로 전용
- **장애 격리** — 한 로봇 PC 문제가 다른 로봇에 전파되지 않음
- **RTDB로 결합** — PC가 달라도 클라우드 DB가 단일 버스 역할

WEB ↔ ROS2

Web → DB → ROS2 — 왜 이 구조인가

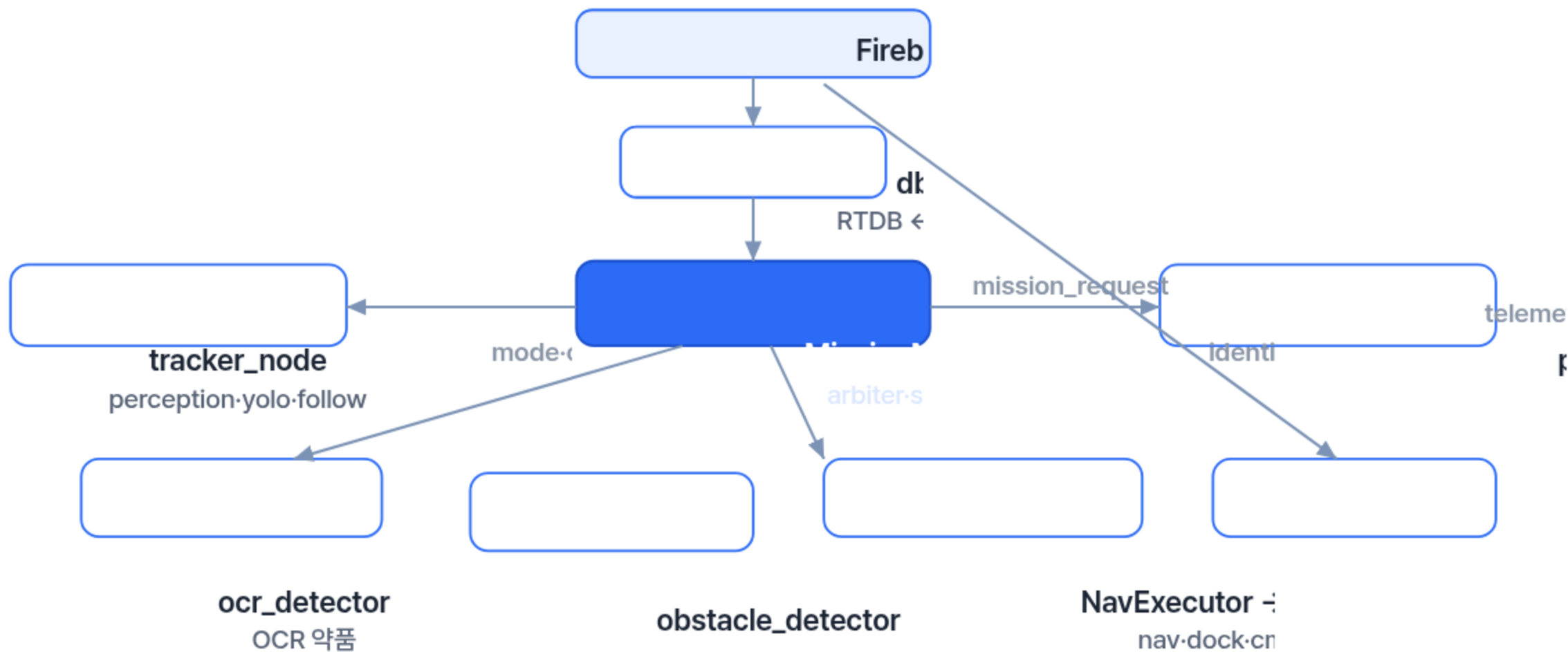


Why — Web→DB→ROS2

- **크로스-PC** — 웹(PC3)과 로봇 PC가 분리, 직접 ROS 연결 대신 클라우드 DB 경유
- **디커플링** — 웹은 RTDB read/write만, 로봇 ROS 노드는 무변경
- **비동기-내구** — 미션 큐가 DB에 남아 재연결/재시작에 강함
- **관측성** — 모든 상태가 DB에 → 대시보드·로그가 단순

ROS2 NODE ARCHITECTURE

ROS2 노드 아키텍처 (per robot)



NODE RESPONSIBILITIES

각 노드가 하는 일

노드 / 패키지	기능
mission_manager	미션 디스패치 · ModeArbiter(cmd_vel 단독·우선순위) · NurseCart/PatrolIntake 시퀀서 · NavExecutor
db_bridge (db_node)	RTDB mission_pool→/ns/mission_request, feedback, ocr_done/round_done/intake_done 브리지
amr_bridge / camera_bridge	amcl_pose·odom·battery·scan·dock → RTDB / annotated·rgb·depth → RTDB
nurse_tracker	perception(RGB+Depth 동기·YOLO) + follow_control(85cm) → cmd_vel
patient_identifier	QR 신원확인(cv2) · patient_validator · PatientIdentified 발행
ocr_detector	약품 텍스트 OCR(EasyOCR/GCP) · medicine_checker · ocr_result
obstacle_detector	depth 기반 정면 장애물 → ground_status
prescription / rooms_server	GetPrescription · ListRooms 서비스(환자·병실 조회)

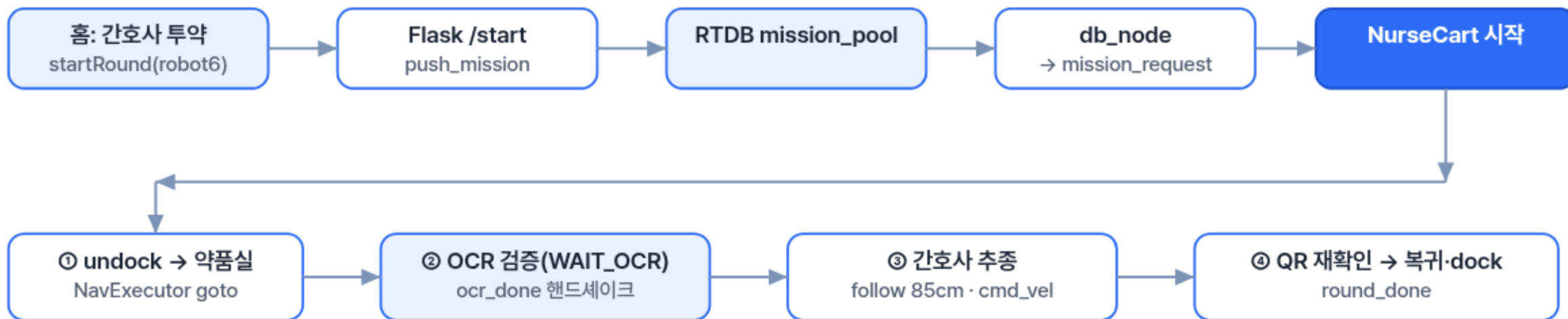
COMMUNICATION STRUCTURE

통신 구조 — 토픽 · 서비스 · 액션

종류	이름 (/ns=robot3-robot6)	메시지 / 용도
Topic	/ns/mission_request · /mission_feedback	String · 미션 요청/피드백
Topic	/ns/mode/<name>/set_cmd_vel-status	String-Twist · 모드 중재(ModeArbiter)
Topic	/ns/cmd_vel · /scan · /amcl_pose · /dock_status	Twist-LaserScan-Pose-DockStatus
Topic	/ns/nurse_cart/ocr_done-round_done · /patrol/intake_done	String · 핸드셰이크(db_node 브리지)
Topic	/ns/identify/start · /patient_identified	String-PatientIdentified · QR 신원
Service	/ns/db/list_rooms · get_prescription · get_ocr_result	ListRooms-GetPrescription-GetOcrResult
Action	/ns/navigate_to_pose · /dock · /undock	Nav2-Create3 · 주행/도킹

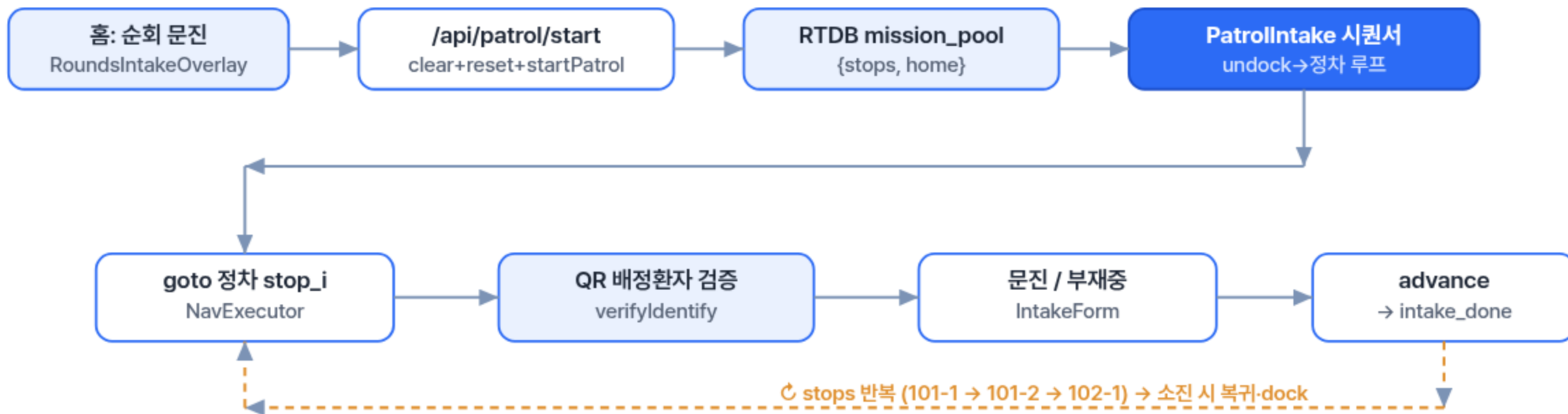
FLOWCHART · 간호사 투약 (robot6)

투약 시나리오 플로우차트



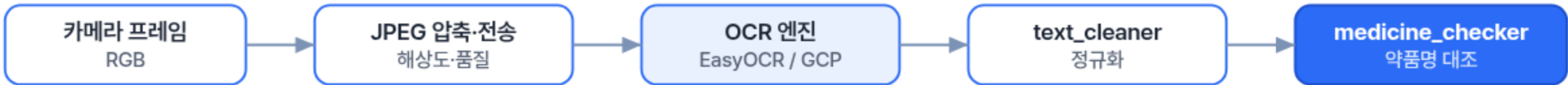
FLOWCHART · 순회 문진 (robot3)

순회 시나리오 플로우차트



OCR · 로직 (input → output)

OCR 노드 파이프라인



OCR 로직 손그림 — ocr_node_flow_v3.svg

여기 OCR 노드 흐름 손그림(input→output)이 들어가야 함

핵심

약품 용기는 외형이 유사 → **인쇄 텍스트로만 구분**. YOLO로는 어려워 OCR로 직접 인식·대조해 **투약 더블 체크**. 결과는 ocr_result/RTDB로 전달돼 시퀀서의 WAIT_OCR을 해제.

OCR · 용도 / 목적

왜 OCR인가 — 약품 식별의 본질

문제

약품 용기는 형태·색이 유사 → 객체 탐지(YOLO)로 정확 식별 곤란.

해결

표면 인쇄 **약품명 텍스트**를 OCR로 직접 인식·대조 → 환자에게 투약할 약을 **더블 체크**.

안전 가치: 오투약을 기계적으로 차단하는 명시적 검증 단계.



약품 용기 비교 사진 (유사 외형 / 텍스트 차이)

실제 약품 용기 촬영 이미지 자리

OCR · 결과 / 모델 비교

EasyOCR vs GCP Vision AI

항목	EasyOCR	GCP Vision AI
비용	무료 오픈소스	유료(호출당 과금)
한글 정확도	다소 낮음(불필요 문자도 인식)	높음 — 깔끔하게 텍스트만
운영	로컬·오프라인·HP 튜닝 가능	인터넷·API 키 필요, 튜닝 불요
채택	—	GCP Vision (정확도 우선)

EasyOCR 인식 결과 스샷
스크린샷 자리 (잡문자 다수)

GCP Vision 인식 결과 스샷
스크린샷 자리 (텍스트만 깔끔)

OCR · 성능 최적화

전송 최적화 — 1.35s → 0.59s

1.35s

기존 (PNG 전송)

0.59s

개선 (JPEG 압축 전송)

방법

PNG 원본 대신 **JPEG 압축** 후 서버 전송 → OCR detection 시간 단축. 해상도·품질을 함께 조정.

측정 구간 (t0 · t1 · t2)

t0- 해상도·JPEG 품질 (인코딩/전처리)

t1

t1- GCP로 이미지 전송 → 텍스트 결과 수신 **왕복 시간**

t2



실시간 OCR detection 시연 GIF

여긴 약품 OCR 인식 시연 영상(gif)이 들어가야 함

QR · 신원 확인 (input → output)

QR — 환자 신원 이중 안전

① robot3 — 순회 문진

병상 도착 시 환자가 QR 제시 → cv2.QRCodeDetector로 patient_id 판독 → Firebase 배경정보 대조
→ 일치/불일치/부재 판정 → patient_identified 발행

② robot6 — 약품 전달 직전

OCR 약품 검증 완료 후 실제 전달 직전 **QR 환자 재확인** → 오투약 방지 이중 장치. Identify/start로 동일
QR 로직 재사용

robot3 QR 체크 — 문진표 흐름

QR→문진 진입 화면/시연

robot6 QR 체크 — 약·환자 확인

약품 전달 직전 환자 확인 화면/시연

YOLO · 모델 비교

YOLO 모델별 성능 비교 → yolov8s



benchmark_comparison.png

YOLO 모델 4종 mAP/속도 비교 그래프 자리

용도/목적

간호사 tracking을 대신해 **RC카 모델**을 학습·검출 → 추종 제어 입력.

결과

모델 4종 비교 결과 **yolov8s 채택** — 정확도/속도 균형 우수.

배움: 실시간 추종엔 mAP뿐 아니라 추론 지연·안정성의 균형이 중요.

DETECTION · 종합 / 배움

Detection 종합 — 무엇을 배웠나

축	채택/결과	핵심 배움
OCR	GCP Vision · JPEG 전송	정확도-비용-지연 트레이드오프, 전송 포맷이 지연을 좌우
QR	cv2 · 2단계 검증	단순·확실한 신원 확인이 안전의 마지막 빗장
YOLO	yolov8s	실시간성=정확도+지연+안정성의 균형



Detection 통합 시연 — OCR+QR 안전 호를 GIF

약품 OCR → 환자 QR 재확인 통합 시연 영상 자리

AMR OVERVIEW

AMR — Mapping · Tracking · Navigation

Mapping

fixed map(ninety) 생성 · costmap 파라미터 튜닝
(inflation·robot radius)

Tracking

YOLO bbox → 좌표변환 → **cmd_vel 직접 제어** (Nav2 goal 아
님)

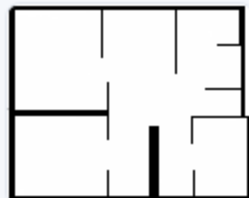
Navigation

Nav2 자율주행 · 정차 순회 · 경로 안정화(101→102 문제 해결)

cmd_vel 단독 소유(ModeArbiter) 위에서 추종/순회가 안전하게 동작 — 모드 간 명령 충돌 없음.

MAPPING · fixed map

맵 생성 — ninety fixed map



방법

slam_toolbox로 병동을 매핑 후 **fixed map(ninety.yaml)**으로 고정 — resolution 0.05, origin 정렬. 깔끔한 맵을 위해 반복 스캔·후처리.

활용

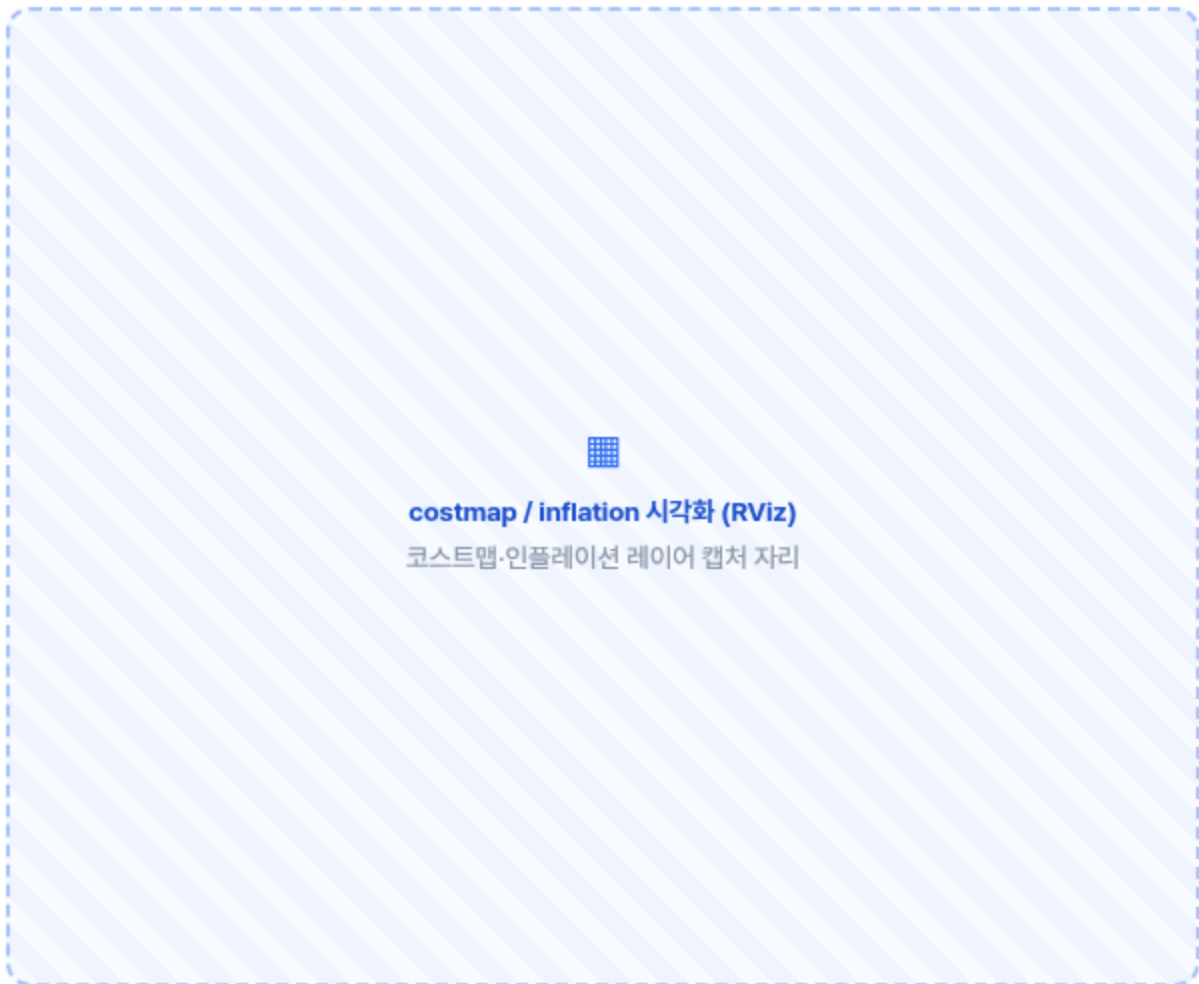
AMCL 로컬라이제이션 + Nav2 글로벌 코스트맵의 기준. 웹 MapView도 같은 mapMeta로 정렬.

맵 생성 과정 캡처(전/후)

깔끔하게 딴 결과 비교 이미지 자리

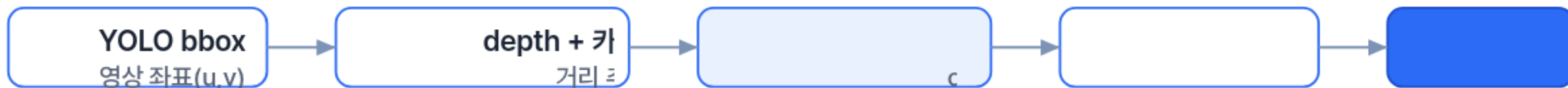
기본 파라미터 — costmap 튜닝

파라미터	값/의도
resolution	0.05 m/px
robot_radius / footprint	로봇 외형 반영 — 충돌 여유
inflation_radius	장애물 주변 비용 팽창 — 경로 여유
cost_scaling_factor	벽 근접 회피 강도
obstacle/voxel layer	LIDAR 기반 동적 장애물 반영



TRACKING · bbox → cmd_vel

추적 — 좌표 변환과 cmd_vel 직접 제어



Why — Nav2 goal이 아니라 cmd_vel 직접 명령?

사람은 연속·동적으로 이동 → 매 프레임 goal 재설정 = 경로 재계획 오버헤드·지연이 큼. 추종은 거리 85cm·각도 오차를 즉시 cmd_vel(전진/회전)로 환산해 반응성을 확보(반응형 제어). Nav2는 정차 이동(goto)에만 사용.

▶ 추종 시연 GIF (간호사/RC카 따라가기)

tracking 동작 영상 자리

TRACKING · 안정화

버벅임 해결 & 도달 시 추적 해제

버벅임(jitter) 해결

bbox/거리 추정 노이즈로 cmd_vel이 진동 → 속도 평활화·데드존·게인 튜닝으로 부드러운 추종.
(cmd_vel 최적화)

도달 시 해제

특정 위치/조건 도달 시 **tracking**을 해제해 불필요한 추종·충돌을 방지하고 안정성↑. 정면
depth<200mm 안전 게이트와 결합.



버벅임 전/후 비교 GIF

최적화 전(진동) vs 후(부드러움) 영상 자리

배움: 인지 노이즈를 제어단에서 흡수해야 실제 추종이 안정적 — 모델 정확도만으로는 부족.

robot3 101→102 — "부비부비" 원인과 해결



문제 상황 GIF (101→102 진입 실패)

로봇이 벽 근처에서 회전 반복하는 영상 자리

증상

101호 → 102호 전환 시 목표 근처에서 전진 못 하고 제자리 회전(부비부비).

원인·해결 (가설 → 검증)

- **inflation/footprint** 과다 → 좁은 통로에서 유효 경로 소실 → 파라미터 조정
- **goal tolerance·회전 비용** 조정 → 진입 각도 완화
- **정차 pose** 재설정 → 벽 여유 확보

순회 주행 로직



Nav2 BasicNavigator

워커 스레드 사용 시 전용 executor로 spin → generator 충돌 방지.

도착·전진 핸드셰이크

도착 phase를 RTDB로 노출 → 웹이 검증/문진 후 advance.

배움

좁은 병동에선 **코스트맵 여유·정차 pose**가 성패를 가름.

COMMUNICATION OPTIMIZATION (optional)

통신 최적화 — 필요할 때만 수신

현재 파이프라인

OAK-D(rgb/depth)·LiDAR·pose가 항상 publish → 대역폭·CPU 부담, 유휴 시 낭비.

최적화 — lazy 구독

구독자가 있을 때만 카메라/스트림을 활성화(OAK-D power-saver-lazy 토픽) → 유휴 시 자동 정지, 추종/OCR 필요 시 재개.

상시 publish (전)

유휴에도 rgb/depth 송출 — 낭비



필요 시만 (후)

구독자 생기면 재개 — 대역폭/전력 절감

효과: 유휴 시 fps=0 → 추종/검증 구간에만 부하 집중. (대시보드 fps가 비는 이유)

SYSTEM MONITOR · UI

모니터링 UI 구성과 설계 의도



대시보드 화면 — 지도·로봇상태·미션 (관리자 콘솔)

여기 웹 대시보드(MapView·AMR 패널·미션) 실제 화면이 들어가야 함

구성

MapView(지도+로봇 pose+도크 오버레이) · **AMR 패널**(배터리·LIVE·경과) · 미션/시나리오 트리거 · 처치실 OCR · 문진/QR.

Why

웹은 ROS 무관, **RTDB read/write만** → 어디서나 접속. SSE로 실시간 반영, 등급별 RBAC로 메뉴 분리.

강조 포인트 (UX · 기술)

실시간 신뢰성

stamp(ms) 기준 LIVE/STALE 판정 — 단위 일치로 오판 제거.

per-robot 분기

robot3/robot6 데이터·홈좌표를 RTDB에서 per-robot로 정확히 표시.

지도 정합

mapMeta(origin·res)로 좌표 변환 → 마커가 맵과 정확히 정렬.

배움

웹 ↔ ROS 사이의 데이터 규약(단위·좌표·타임스탬프)이 곧 신뢰성 — 시각화 품질은 규약 정합에서 나온다. SSE·RBAC로 운영 편의와 보안을 동시에 확보.

INTEGRATION TEST · 전략

시스템 통합 테스트 전략

접근

단위 기능 테스트 대신 시나리오 단위 시스템 통합 테스트로 검증(웹 레이어는 자동화 테스트 별도 보유).

정상 동작 확인 포인트

RTDB `mission_pool·phase` 핸드셰이크 플래그, ROS `mission_request/feedback`, `cmd_vel·dock_status` 로그.

확인할 데이터/로그

- RTDB: `{ns}/mission_pool`, `nurse_cart/patrol/phase`, `ocr_done·round_done·intake_done`
- ROS: `/[{ns}]/mission_request·feedback`, `/[{ns}]/cmd_vel`, `/dock_status`, `/scan`
- Web: SSE 스트림, `/api/amrs·patrol·nurse_cart` 응답

TEST CASE CATALOG

테스트 케이스 목록 (카탈로그 98)

영역	케이스	대표 연결부
공통 (X)	28	RTDB·stamp(ms)·mission_pool · RBAC · telemetry · SSE · Map
간호사 투약 (A)	36	트리거·시퀀서·OCR·인지(RGB/Depth/YOLO)·추종85cm·핸드셰이크
순회 문진 (B)	34	발행·순회·Nav2·QR 배정검증·문진/부재중·patrol phase/advance

98

카탈로그 케이스

137

웹 자동화 통과 (프론트66+백엔드71)

참고 문서

docs/superpowers/specs/
2026-06-10-scenario-test-case-catalog-design.md

INTEGRATION TEST · 결과

통합 테스트 결과 — 시나리오 시연



간호사 투약 전체 시나리오 시연 GIF

약품실→OCR→추증→QR→복귀 통합 영상 자리



순회 문진 전체 시나리오 시연 GIF

정차→QR검증→문진→advance→복귀 통합 영상 자리

결과: 두 시나리오 end-to-end 동작 확인 · 웹 자동화 137/137 통과. (잔여: advance 디바운스·문진 검증은 후속 — 5장 평가 참고)

RGB 기반 Depth 추론 — 왜?

동기

Depth 센서 의존을 줄이고 **RGB만으로** 바닥 평면상의 물체·사람까지의 실제 거리를 추정하고자 함.

원리 (개요)

- ① 바닥 4개 기준점 ↔ 실제 좌표 대응 → **Homography H** 계산
- ② YOLO 사람 발목 좌표(u,v)를 H로 변환 → 바닥 좌표(X,Y)
- ③ **Bird Eye View**로 위치·경로·거리 계산



개념도 — 카메라 영상 → Homography → BEV

구조도/원리 이미지 자리 (Image.png)

Homography 행렬 H — 픽셀 → 바닥 좌표

변환식

```
# 3x3 Homography H
 $[X \ Y \ 1]^T \propto H \cdot [u \ v \ 1]^T$ 
# 입력 (u,v) = 영상 속 사람 발목 좌표
# 출력 (X,Y) = 실제 바닥 좌표 (BEV)
```

의미

카메라 픽셀 좌표를 실제 바닥 평면 좌표로 변환하는 행렬 → 상공에서 내려다본 **Bird Eye View** 생성.
위치·이동경로(Trajectory)·사람 간 거리를 실제 공간 기준으로 계산.



Homography / BEV 시각화

입력 영상 + 변환 후 BEV 이미지(image.png) 자리

R&D · 결과 (오차) / 배움

거리 추정 결과 — 25.8% → 6.0%

회차	실측값(m)	측정값(m)	오차(m)	오차율	비고
1	3.64	2.70	0.94	25.82%	초기(오차 최대)
2	4.83	4.56	0.27	5.59%	1차 보정
3	3.12	2.97	0.15	5.00%	안정화 진입
4	1.82	1.71	0.11	6.04%	최종

측정 셋업 / BEV 결과 캡처
실험 장면·BEV 출력 이미지 자리

결과

보정 반복으로 오차율 25.8% → 5~6%대로 안정화.

배움: 기준점 선택·평면 가정의 정확도가 오차를 좌우 — 단일 평면(바닥) 가정의 한계도 확인.

간호사는 지금 너무 많은 일을 한다

업무 과부하

간호사 1인당 환자 과다 → 단순 문의 응대에도 시간 소모

언어 장벽

외국인 환자 증가 → 소통 지연

호출 체계

콜벨·수기 → 호출 누락·우선순위 불가

약품 관리

수기 기록 → 오투약 리스크

핵심 메시지

로봇이 움직이려면 **정보 인터페이스**가 반드시 필요하다 — 키오스크는 로봇의 입·귀, 직원 대시보드는 로봇-간호사 중계. 로봇의 실제 유용성은 이 앱이 완성한다.

왜 이 앱이 필요한가 — 기능 ↔ 근거

기능	근거
AI 챗봇 (Ollama)	간호사가 반복 응대하던 위치·진료 문의를 자동화
4개국어 + 음성 I/O	외국인·고령자 스스로 해결 → 간호사 호출 감소
실시간 BroadcastChannel	로봇 호출·간호사 호출을 즉시 대시보드에 표시
QR 약품 관리	수기 오류 제거, 투약 이력 추적
PWA / 오프라인	병원 내 불안정 네트워크에서도 동작 보장

기대 ①

간호사 단순업무 시간 감소 → 핵심 의료에 집중

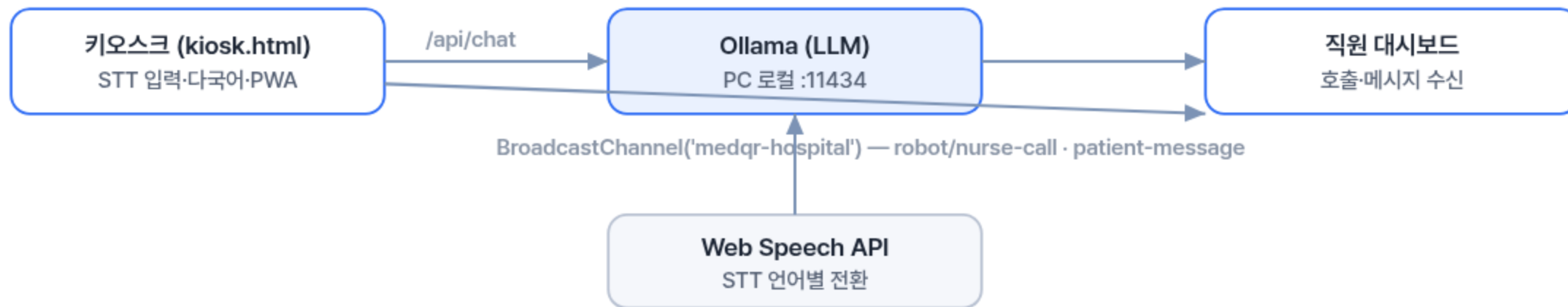
기대 ②

환자 자기서비스 → 대기 단축·만족도 ↑

기대 ③

로봇-앱-간호사 3자 연동 → 현장 투입 가능

STT 정보 웹 — 구조



백엔드 서버·WebSocket 없이 같은 PC 탭 간 BroadcastChannel로 실시간 중계 — 인프라 없이 즉시 구현.

R&D · STT Web · APK → 웹 전환 & 핵심 코드

왜 APK를 버리고 웹으로 갔나

근본 원인

APK에서 **localhost:11434** 호출 → 안드로이드 localhost는 폰 자신을 가리킴 → Ollama(PC)엔 도달 못 함 → failed to fetch. (Gemini는 클라우드 HTTPS라 무관)

웹 전환으로 해결

웹 서버를 PC에서 띄우면 **location.hostname = PC IP** → 같은 Wi-Fi 모든 기기가 Ollama 호스트를 자동·정확히 지정.

핵심 코드 — Ollama 호스트 자동 감지 (kiosk.html:292)

```
const OLLAMA_HOST = (location.hostname === 'localhost' || location.hostname === '127.0.0.1')
  ? 'localhost' : location.hostname;
const OLLAMA_URL = `http://${OLLAMA_HOST}:11434/api/chat`;
// PC에서 열면 localhost, 모바일에서 열면 PC IP 자동 사용 - 추가 설정 불필요
```

그 외: 대화 히스토리 20턴 유지 · 언어별 시스템 프롬프트 교체 · STT 언어 자동 전환(ko/en/zh/ja)

TensorRT — 고성능 추론 가속

세팅

학습된 YOLO 가중치를 **TensorRT 엔진으로 변환**(FP16) → GPU 최적화 추론. 환경:
CUDA·TensorRT·trtexec.

실험

PyTorch vs TensorRT 추론 **지연·FPS 비교** → 실시간 추종/검출에 적합한지 검증.



TensorRT 가속 전/후 FPS·지연 비교 그래프

벤치마크 결과 이미지 자리

R&D · 종합 / 배움

Ninetygale R&D 종합

주제	핵심	배움
Depth Inference	Homography-BEV, 오차 6%대	기준점/평면 가정이 정확도를 좌우
STT Information Web	키오스크·Ollama·BroadcastChannel	로봇 유용성은 정보 인터페이스가 완성, 네트워크 토폴로지 이해
TensorRT	엔진 변환·추론 가속	실시간성엔 모델뿐 아니라 런타임 최적화가 필요

SELF-EVALUATION

자체 평가 의견

성과

RTDB 단일 버스 아키텍처로 두 시나리오 end-to-end 통합 ·
OCR/QR 이중 안전 · 웹 자동화 137 통과 · 지식그래프(499노드)로
구조 가시화.

한계

advance 디바운스·문진 폼 검증 미구현(현재 동작 한계 식별) ·
Depth 단일 평면 가정 한계 · 단위 테스트는 일부 후속.

향후

ROS 단위/통합 자동화(Plan 2·3) · 문진 UX(드롭다운·검증) · 다로
봇 확장 · TensorRT 본격 적용.

총평: "로봇만"이 아니라 **로봇-DB-웹-앱**을 하나의 안전한 워크플로우로 묶어 현장 투입 가능성을 입증.

THANK YOU

병동 보조 로봇 **MediCart** — 끝까지 봐주셔서 감사합니다.
두 대의 AMR로 간호사 투약과 순회 문진을 자동화한 풀스택 프로젝트였습니다.

레퍼런스: `final_report.pptx` · `docs/architecture/diagrams` · 지식그래프(.understand-anything)